LINUX LAB | PROJECT REPORT

Metro Fare Calculator

L55 - Case, AWK, BC, Sort & Cron

STUDENT	PROJECT INFORMATION
Student	HADI MOMIN
Roll / Division	F20 / G
Enrollment	24004501210207
System	Ubuntu / WSL Ubuntu - Bash shell
Project path	~/linux_project/Hadi/farecalc

Problem statement and objective

Create a menu-driven terminal utility to estimate sample metro fares between listed stations, save journey history, calculate monthly travel cost, compare it with a monthly pass, and report the most-used route. Intended for a coursework demonstration of Linux shell scripting, not for issuing tickets.

Required project deliverables

Requirement	Implemented behavior
Five interactive features	Fare quote, log journey, monthly totals, pass comparison and most frequent route
Data	16 station records and 12 sample journeys; prices stored with journeys
Shell commands	case, awk, bc, sort, uniq -c, tar, date and crontab
Automation	Month-end comparison report at 9 PM on the final calendar day
Supporting outputs	README, archive backup, logs, command-line options and tests

Data disclaimer: Distances and fare slabs are illustrative classroom values. The example pass price is INR 800.00. They are not verified official Ahmedabad Metro fares or pass offerings.

Architecture & Core Logic

Project folder structure

```
farecalc/
  main.sh               Menu and CLI command dispatch
  lib/functions.sh      Fare, journey, monthly report functions
  data/stations.csv     station,distance_km (16 sample stations)
  data/journeys.csv     date,from,to,fare
  data/pass_price.txt   Example monthly pass price
  reports/              Report files from --cron
  backups/              Data snapshots (.tar.gz)
  logs/activity.log     Time-stamped action history
  tests/run_tests.sh    Isolated test copy of project
  README.md / CRON_GUIDE.md
```

Sample fares used for the demonstration

Distance interval	Illustrative fare	Bash mapping
0-2 km	INR 5.00	short
Over 2-5 km	INR 10.00	local
Over 5-10 km	INR 15.00	medium
Over 10-15 km	INR 20.00	long
Over 15 km	INR 25.00	extended

How the five features work

1. Fare by distance slabs: The script searches for matching station names with awk, calculates the absolute difference between their sample positions using bc, assigns a category, then uses a Bash case statement to choose the fare.

2. Log a journey: It validates stations and an ISO date, calculates the fare, and appends date, from, to, fare to journeys.csv.

3. Monthly travel cost: awk filters journey dates by YYYY-MM and sums the saved fares. The output shows ride count, total cost, and the journeys.

4. Compare with pass: bc compares the accumulated ticket expense to the configurable sample monthly pass price and explains the cheaper option.

5. Most frequent route: awk extracts the station pairs, and sort | uniq -c | sort -nr ranks the directed routes.

Demonstration Results

The examples below are terminal output captured from an isolated automated test environment. They are not screenshots of the student's own WSL machine, and they do not establish that cron ran on schedule.

Example A - Fare estimate, monthly cost and pass comparison

```
$ ./main.sh --fare Vastral Thaltej
From: Vastral               To: Thaltej
Sample distance: 21.0 km
Illustrative single fare: INR 25.00

$ ./main.sh --monthly 2026-10
MONTHLY TRAVEL REPORT - 2026-10
Journeys: 10
Total spent: INR 195.00

$ ./main.sh --compare 2026-10
Month: 2026-10
Journeys: 10
Single tickets total: INR 195.00
Illustrative monthly pass: INR 800.00
Recommendation: Single tickets are cheaper by INR 605.00
```

Example B - Most frequently travelled route

```
$ ./main.sh --frequent
MOST FREQUENT DIRECTED ROUTES
      5 Vastral -> Thaltej
      2 Vastral -> Kalupur
      1 Kalupur -> Gheekanta
      1 Kalupur -> Thaltej
```

Validation and automated testing

Verification	Result
Core fare slabs / reversed journeys / station validation	PASS - 5 slab values, reverse lookup, bad station
Journey data and date validation	PASS - valid save and invalid-date rejection
Monthly totals and pass decisions	PASS - expense aggregation and comparison
Menu, backups, cron report output, logs	PASS - interactive features and report files
Isolated automated test suite	23 checks passed using an arithmetic compatibility shim

Environment limitation: GNU bc was unavailable in the report-generation environment, so the automated logic checks used a test-only arithmetic substitute. Run `sudo apt install bc` and repeat the suite on actual Ubuntu before your viva.

Cron, Submission & Viva

Run and test on WSL Ubuntu

```
cd ~/linux_project/Hadi/farecalc
sudo apt install bc cron
./tests/run_tests.sh
./main.sh               # choose options 1 to 9
./main.sh --cron 2026-10
./main.sh --backup
cat reports/monthly_fare_vs_pass.txt
cat logs/activity.log
```

Required cron schedule: 9 PM on the last day of every month

Start cron with sudo service cron start; open crontab -e (not sudo) and add the line below. In cron syntax, \% must be escaped. Keep other students' cron entries unchanged.

```
0 21 28-31 * * [ "$(date -d tomorrow +\%d)" = "01" ] && \
/bin/bash "$HOME/linux_project/Hadi/farecalc/main.sh" --cron \
>> "$HOME/linux_project/Hadi/farecalc/logs/cron.log" 2>&1
```

The date test restricts execution to the last day of the month. This task only runs if the WSL environment and cron service are active; save actual crontab -l and cron log screenshots as evidence.

Submission checklist

Include F20_L55_farecalc.tar.gz (project files, at least 15 stations, reports and scripts), plus this 4-page PDF. Take your own Ubuntu screenshots of the menu, five options, backup, logs, and cron schedule/execution; the supplied examples are not substitutes for real screenshots.

Viva questions and short answers

What is a shebang? #!/usr/bin/env bash selects the interpreter that runs the file.

Why case? It picks the sample fare corresponding to a distance class and dispatches menu choices.

What is awk used for? Selecting station records and filtering/summing trip expenses by month.

Why bc? Bash arithmetic alone cannot directly handle decimal distances and costs.

What does uniq -c do? It counts consecutive repeated routes after they are sorted.

What does tar -czf do? Creates a gzip-compressed archive of data files.

How is cron scheduled? The five timing fields specify 9 PM on possible month-end dates, followed by a check for the actual final day.

What I learned: Shell functions, CSV processing, input validation, decimal arithmetic, logs, backups, reporting and cron automation. Personalize this explanation before submission.